20

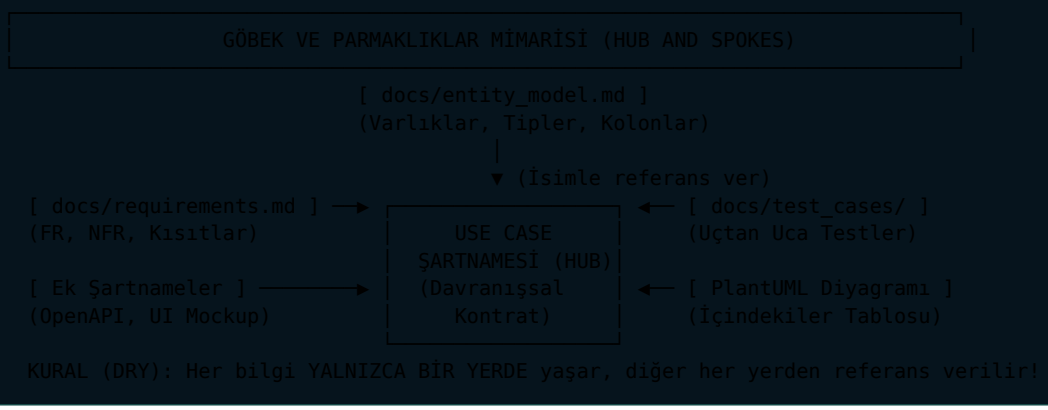
BÖLÜM: ESER ZINCIRI (ARTIFACT CHAIN) & PLANTUML MIMARISI | MODULE: THE ARTIFACT CHAIN & PLANTUML ARCHITECTURE

Göbek ve Parmaklıklar Modeli (Hub & Spokes): Use Case Neden Sadece 'Bölüm 3'tür?

The Hub-and-Spokes Model: Why Use Cases Are Chapter 3 of the Requirements

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir bisiklet tekerleğini düşünün: Ortada sağlam bir tekerlek göbeği (Hub) vardır ve bu göbekten dışarıya doğru onlarca metal parmaklık (Spokes) uzanır. Use Case bu tekerleğin tam merkezidir; ama tek başına gidemez! Bir parmaklık sözlüğe (Entity Model), diğer parmaklık hız sınırına (NFR), öbürü testlere gider. Eğer göbeğin içine lastiği, freni ve pedalları tıkıştırmaya çalışırsan tekerlek dönmez! Use case sadece 'kim ne yaptı' hikayesini anlatır; teknik detaylar parmaklıkların ucundaki kendi evlerinde yaşar.

🇬🇧 ENGLISH (ELI5)

Visualize a bicycle wheel: at the center sits the structural hub, with metallic spokes radiating outward to support the rim. The use case specification is the central hub. It governs behavioral flow, but delegates non-behavioral concerns out along its spokes: data definitions point to the Entity Model, performance budgets point to the NFR table, and user journeys point to Test Cases. Jamming database tables and UI wireframes directly into the hub fractures the wheel.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Cockburn'ün 'Bölüm 3' Metaforu ve DRY Prensibi: Cockburn, kullanım senaryolarının gereksinim dökümanının tamamı olmadığını, sadece 'Bölüm 3: Davranışsal Gereksinimler' olduğunu söyler. Diğer tüm konular use case'in dışındaki özel dosyalarda yaşar: 1) Veri Yapısı ve Alan Tipleri: `docs/entity_model.md` içinde tanımlanır; use case sadece 'Üye' veya 'Ödünç' ismini kullanır, asla içinde tablo şeması veya kolon listesi barındırmaz. 2) NFR ve Kısıtlar: `docs/requirements.md` içindeki NFR tablosunda yaşar; use case sadece `Requirements: [FR-012, NFR-002]` olarak linkler. 3) UI Mockup'ları ve OpenAPI: Ek dökümanlar olarak bağlanır. 4) Bilgi Tekrarı Yasağı (DRY): İnsan geliştiriciler için çift yazım sadece bir bakım zorluğuydu. Ancak yapay zekâ için çift yazım ÖLÜMCÜLDÜR! İki döküman zamanla birbirinden farklılaşır ve aralarında çelişki doğar. Bir AI ajanı çelişki gördüğünde bunu bir soru olarak sormaz; kafasına göre bir tarafı seçerek kod üretir. Bu yüzden her bilgi tam olarak TEK bir yerde tanımlanır ve sadece referans verilir.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Cockburn's 'Chapter 3' Doctrine and Requirements-Level DRY: Cockburn famously noted that use cases constitute merely Chapter 3 of a complete requirements document: the behavioral slice. Everything that resists the narrative structure is spun off into dedicated satellites. In the AI Unified Process, these spokes are mechanically formalized: 1) Data Schemas live in `docs/entity_model.md`. The spec cites domain nouns by business name and never inlines attribute tables. 2) Non-functional constraints reside in `docs/requirements.md` and are referenced via metadata tags. 3) Contract interfaces (OpenAPI, UI wireframes) are hyperlinked supplementary files. 4) The AI-Era DRY Imperative: For human teams, duplicated requirements created synchronization debt. For LLMs, duplication produces catastrophic failure: when two documents diverge by a single field, the AI resolves the contradiction silently, picking an arbitrary side. DRY is no longer merely a coding aesthetic; it is an existential requirements invariant.

⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Hub & Spokes Prensibine Göre Ayrıştırılmış Temiz Domain Mimarisi
package com.maang.aiup.hubspokes;

// SPOKE 1: Domain Modeli (docs/entity_model.md karşılığı)
record Member(String id, String name, String email) {}
record BookCopy(String copyId, String isbn) {}

// SPOKE 2: İş Kuralı Portu (docs/requirements.md karşılığı)
interface LoanPolicyGate {
    boolean canBorrow(Member member);
}

// HUB: Use Case İcra Sınıfı (docs/use_cases/UC-002 karşılığı - Sadece Akışı Yönetir!)
public class BorrowBookHubUseCase {
    private final LoanPolicyGate policyGate;

    public BorrowBookHubUseCase(LoanPolicyGate policyGate) {
        this.policyGate = policyGate;
    }

    public void execute(Member member, BookCopy copy) {
        // Hub sadece orkestrasyon yapar, veritabanı veya karmaşık kural detaylarını spokelara delege
eder
        if (!policyGate.canBorrow(member)) {
            throw new IllegalStateException("Ödünç alma kuralı ihlal edildi!");
        }
        System.out.printf("Hub tamamlandı: %s üyesine %s kitabı tahsis edildi.%n", member.name(),
copy.isbn());
    }

    void main() {
        var hub = new BorrowBookHubUseCase(_ -> true);
        hub.execute(new Member("M1", "Ada Lovelace", "ada@lovelace.org"), new BookCopy("C1",
"978-01"));
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Use case şartnamesinin içine SQL 'CREATE TABLE' komutları veya ekranın piksel genişliklerini ASLA yazmayın! Bu detaylar parmaklıkların ucundaki varlık modeline ve UI mockup dosyalarına aittir. Göbeği şişirmek AI'ın aklını karıştırır.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never embed DDL statements (`CREATE TABLE`), column data types, or pixel coordinates directly inside a use case scenario. Inlining schema definitions bloats the hub and invites silent synchronization drift.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• Use Case tekerleğin göbeğidir; sadece davranışsal akışı yönetir, detayları spokelara devreder.

• Veri yapıları entity_model.md'de, performans NFR tablosunda, testler ise test_cases klasöründe yaşar.

• DRY artık sadece kodlama değil, şartname kuralıdır: Çelişkili iki bilgi AI'ın sessizce yanlış kod yazmasına yol açar.

🎯 Key Architectural Takeaways

• The use case is the structural hub; it orchestrates behavior while delegating details to spokes.

• Data schemas live in the entity model, NFRs in the catalog, and journeys in test cases.

• DRY is a requirements imperative: contradictions across duplicated specs cause silent AI bugs.